

ECE 411

Spring 2026

Review Notes for ECE 411 (Computer Organization and Design)

These notes aren't fully comprehensive.

Aniketh Tarikonda (aniketh8@illinois.edu)

Notes based off of:

- Lectures by Prof. Kumar
- *Computer Organization and Design: The Hardware/Software Interface, 5th Edition, David Patterson and John Hennessy*
- *Computer Architecture: A Quantitative Approach, 5th Edition, David Patterson and John Hennessy*
- *Processor Microarchitecture: An Implementation Perspective*

Contents

1	Intro	1
2	Pipelining	2
2.1	Data Hazards	3
2.1.1	Forwarding	3
2.2	Branch Hazards	4
2.2.1	Branch Predictor (BP)	5
3	Caches	6
3.1	Cache Organization	7
3.2	Cache - Virtual Memory Interactions	10
3.2.1	Address Translation	10
4	Out-Of-Order Execution	12
4.1	Core Components and Stages	12
4.2	Tomasulo's Algorithm (with ROB)	15
4.3	Precise Exceptions	15
5	Memory Systems	16
5.1	DRAM	16
5.2	Load-Store Queue (LSQ)	17
6	Multicore Systems and Multithreading	18
6.1	Multi-Core Systems	19
6.2	Multithreading	19
7	Memory Consistency and Cache Coherence	20
7.0.1	Program Order & Memory Consistency	20
7.1	Cache Coherence	21
7.1.1	Snoopy Bus versus Directory-based	21
7.1.2	MSI	21
7.1.3	MESI	21
7.1.4	MOESI	22
8	Miscellaneous	22
8.1	Interconnects	22
8.2	VLIW	23
8.3	Power	23

1 Intro

- Von Neumann Architecture - Instructions and Data are treated the same.
 - most basic computer architecture, LC-3 is based off of a classical Von Neumann Architecture
 - also has microsequencers
- Harvard Architecture - Separate instruction and data memories
 - used in DSPs today
- Dataflow Architecture - less “sequential”, instructions are sent for execution the moment their operands are ready.
 - while a strict dataflow architecture isn’t commonly used today, it rears its head through dynamic scheduling/out-of-ordered execution.
 - requires a different programming paradigm - more “graphical” programs, with operations being the nodes
- A typical CPU follows Fetch → Decode → Execute → Memory → Writeback

Instruction Set Architecture (ISA) - the contract/interface between hardware and software

- A good ISA is portable, general, provides convenient functionality to higher levels, allows an efficient implementation in lower levels
- Semantic Gap: How close are instructions, data types, addressing modes to the high-level language?
 - Who is the burden being placed upon? The hardware designer or the software designer?
 - You can somewhat “virtualize” ISA - having separate ISAs for the hardware and software, and a translator between the two. Ex: Rosetta on Apple Silicon
- Gray areas exist, x86 is so dominant that it can get around nightmare-ish decode logic. Legacy ISAs still exist because the cost of migrating to newer ISAs outweighs the benefits.
- Using a licensed ISA is often expensive (cough cough arm), but often more commercially viable than using custom ISAs.
- RISC vs CISC - RISC allows for simpler hardware implementations (generally), CISC can have better code density.
 - Modern CISC processors (e.g. x86) use dynamic translation to generate RISC-like μ -ops anyways, so this debate is kinda pointless today.
- ISAs don’t have to be strictly register-register (although register-register is pretty much exclusively used today)–you can have accumulator/stack-based architectures.
- Number of (Architectural) Registers is entirely dependent on ISA
 - More registers allow better register allocation, less compiler tricks, fewer saves/restores
 - More registers means larger instruction size, and larger area penalties

- Indirection: We can do a lot of funky stuff at the hardware level, and as long as the interface (ISA) is being upheld, its all good.
 - Therein lies the difference between ISA and Microarchitecture.
 - E.g. **add** instruction vs. adder implementation (CLA, CRA, hierarchical, etc.)

Big Endian vs Little Endian

- Big Endian: MSB at lowest address
- Little Endian: LSB at lowest address
- RISC-V is Little Endian.

How do we quantify performance?

- In general, performance is given by:

$$\frac{\text{instructions}}{\text{program}} \times \frac{\text{cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}} \quad (1)$$

Different terms in this equations are governed by different levels of abstraction. $\frac{\text{instructions}}{\text{program}}$ is dependent on the programmer. $\frac{\text{cycles}}{\text{instruction}}$ is dependent on microarchitecture and system architecture. $\frac{\text{seconds}}{\text{cycle}}$ is dependent on microarchitecture, pipeline depth, circuit design, semiconductor/VLSI technologies.

Multicycle vs Single-Cycle Processors

- f_{clk} of a single-cycle processor is limited by the longest instruction (often load/stores)
- f_{clk} of a multi-cycle processor will often be a lot faster, although the time taken for the longest instruction will often be longer (because of additional register overheads).
- Although multi-cycle processor is a lot better, it suffers from hardware underutilization and throughput issues. If only there was a way to solve both of these issues...

2 Pipelining

- improves throughput, hardware utilization, latency of instructions stays (mostly) the same.
- potential speedup correlated with the number of pipeline stages
 - with more pipeline stages, the number of cycles required to saturate the pipeline increases.
- each instruction takes a uniform number of cycles to pass through a pipelined processor.
- Pipelining introduces a whole class of issues: **hazards**

There are three types of hazards

- Structural Hazard: a hardware resource is currently busy
- Data Hazard: Need to wait for a previous instruction to read/write
 - RAW (read-after-write) is a true data dependency
 - WAW/WAR are false dependencies
- Control Hazards: branches, jumps

2.1 Data Hazards

- One method of eliminating data hazards in a pipelined processor is to increase the distance between dependent instructions, e.g. by inserting nops between them.
 - introduces performance, portability, code density penalties
 - programmer/compiler needs to be aware of the exact microarchitecture so they can insert the correct number of nops, can we really trust them?
- Rather than inserting nops, you can also re-order code to increase distance between dependent instructions. This is annoying to do however, both for the compiler and the programmer.
- We can try *stalling* once a data dependency is detected. This is better in some aspects than manually inserting nops, but still has poor performance.
 - Method 1: Register Dependency Table
 - When an instruction modifies a particular rd , keep track of it in a dependency table (flip a used bit to 1).
 - Subsequent instructions look at the table, stall if one of their registers is currently being used.
 - When an instruction commits, flip the rd 's used bit to 0
 - Issue: enforces naming hazards, sometimes instructions that don't have strict data dependencies are forced to stall because of shared rd
- The better approach to data hazards: **Forwarding**

2.1.1 Forwarding

- also called data bypassing, forwarding the result of a previous instruction to an earlier stage.
 - ignore values read from register file - they contain values from a previous architectural state
- Implementationally, we add muxes to the EX-stage, WB-stage¹.
 - Control logic for the muxes are generated by a forwarding unit.
- Forwarding unit can be implemented with a priority encoder
 - Pipeline stages closer to EX given priority over those further away.
 - If rd matches select forwarded data from that particular pipeline stage.
- Extra considerations from mp_pipeline:
 - Memory instructions require stalls & forwarding.
 - Load-Use Hazards: when forwarding from the Data Memory stage, data has to travel from SRAM array of Data Cache through forwarding logic, through ALU until it hits the next register. **This is often a critical path**
 - Solution: insert a bubble when a load-use is detected, use the WB forwarding path instead.

¹not always necessary, depends on implementation

- Detect load-use early, in the decode stage.

2.2 Branch Hazards

- Naive solution: stall until branch resolution before issuing instructions
- By shifting branch resolution earlier in the pipeline, you can reduce branch-related penalties.
- Speculative Execution: What if we speculatively execute instructions based on predicted branch direction, and flush if we happen to make an incorrect guess?
- High-performance cores need to sustain 1 instruction fetched per cycle
 - this seems alright, until we consider branch/jmp/return from subroutine instructions
 - these instructions require special prediction units/speculative execution
- Fetch address prediction works in two steps
 - predicting direction of branch (taken/not-taken) - this is done with a **Branch Predictor**
 - predicting target address of branch - this is done with a **Branch Target Buffer (BTB)**
 - some processors treat return from subroutine as a special case, keeping track of these using a **return address stack (RAS)**

Branch Target Buffer (BTB)

- Most branch instructions have PC-relative offsets, which means we could just calculate the target address with an adder (PC + offset)
 - we would have to then have a separate pipeline stage (within fetch unit) for this adder
 - for high-performance, high-frequency processors this would incur a 1-cycle bubble every time a branch is taken (bad for performance)
- Thus, there is value in predicting target addresses with a BTB
- The BTB is normally implemented as a cache; it consists of a table indexed by fetch address, only containing a target address if the specified entry is a branch instruction
- The prediction is the target address as it was executed previously.

Return Address Stack (RAS)

- BTB could predict returns from subroutine pretty well, but RAS is simpler and has even more accuracy.
- RAS is a stack structure (LIFO); when a subroutine call is fetched, the address of the next instruction is pushed into the stack.
- When a return instruction is fetched, the most recent (top of stack) entry is popped and used as the target address.
- RAS is small, only consisting of a few tens of entries.
 - when the RAS is full, the oldest (bottom of stack) entry is overwritten
 - subroutine nesting level $\leq$ RAS size ...generally (the exception when we have deep recursive subroutines)

2.2.1 Branch Predictor (BP)

- Flushing pipelines can incur a significant performance penalty, so we want really good BPs
- Static branch prediction has the smallest hardware overhead, but has the worst performance and requires profiling the code beforehand to see high-level trends in branch direction.
 - ex: mp_pipeline, static NT leads to best-case IPC on CoreMark of ~ 0.7
- All modern processors have Dynamic BPs instead.

2.2.1.1 Local BP

- Consists of a table containing 2^N entries and 2 bits per entry, where N is chosen as the N least-significant bits of the instruction address.
- The 2 bits represent the four states of a saturating counter FSM (Strongly NT $\rightarrow$ Weakly NT $\rightarrow$ Weakly Taken $\rightarrow$ Strongly Taken)
- Called ‘local’ as the decision of one branch doesn’t effect the other.
- Large programs can have aliasing (multiple addresses mapping to one entry in the BP) causing degradation in performance.
- Sometimes adequate, often times not. Accuracy generally ranges from 80% to 99%

2.2.1.2 Correlating BP and gshare

- Similar to local BP, but add a register (Branch Global History) that stores the outcome of the most recent branch instructions (10-20 past outcomes suffices, usually).
- This Branch Global History (BGH) is combined with PC through a hashing function, generating an index for a table consisting of 2-bit saturating counters.
 - this approach is called **gshare**
 - its been found that bitwise XOR between BGH and the least-significant bits of PC is a simple/effective hashing method which minimizes aliasing.

2.2.1.3 Two-Level BP

- Keep both a Branch History Table (BHT) and a Prediction History Table (PHT)
- PC can directly index into BHT or we can have some intermediate hashing.
- BHT keeps track of previous branch results, PHT is indexed by BHT, contains predictions of branch outcomes.

2.2.1.4 Hybrid BP

- Local BP has a shorter warm-up time than Correlating BP/gshare. Thus, in certain scenarios (e.g. right after a context switch), it may be beneficial to initially use Local BP, and later swap to a Correlating BP.
- Implementation-wise, consists of two branch predictors (local and correlating) and a selector, itself behaving like a branch predictor.
 - the select can be implemented as a table of 2-bit saturating counters where the MSB determines which BP to use.
 - selector drives a MUX, which selects the choice of BP to use.

2.2.1.5 Challenges to Branch Prediction

- Aliasing: can be destructive, or constructive (such as with multithreading)
- Training Time: Some branch prediction schemes require a long warm-up time.
- Context Switches
- Timing: In order to meet timing, the branch predictor itself needs to be thoroughly pipelined.

Instruction Cache Considerations

- Usually VIPT or PIPT
- Usually set-associative
- Superscalar designs generally read consecutive bytes from the same cache line
 - decoding logic is trivial for fixed-size RISC instructions
- Conventionally, instructions are stored in the same order as in the binary
 - trace caches store instructions in dynamic order

3 Caches

- Important aspect of memory hierarchy; hides long DRAM latencies due to program memory generally exhibiting **spatial and temporal locality**
 - Spatial Locality: If I access address A , I'm likely to access address its neighbors ($A + 4$, $A + 8$, etc.)
 - Temporal Locality: If I access an address repeatedly, I'm likely going to access it again.
- Organized in hierarchies (Level 1 to Level 3, usually)
- Lower level caches are closer to core, have lower associativity, store less data, and have the lowest nominal latency; L2/L3 caches are larger, hold both instructions/program data
 - L2/L3 caches can be 16, 32+ way set associative because we want to minimize **conflict misses**
 - Serial vs. Parallel Accesses of Cache Levels: We can speculatively access L2/L3 when accessing L1, thus reducing latency on misses. However, this adds unnecessary accesses to next levels on hits.
 - Over a long time, different caches exploit different memory access patterns
- In general, instruction cache (icache) access is simpler than data cache (dcache) because of memory disambiguation logic necessary with data cache.
- How do we evaluate latency of these memory systems?
 - **AMAT**, or Average Memory Access Time

$$\text{AMAT} = (\text{Hit \%})(\text{Hit Latency}) + (\text{Miss \%})(\text{Miss Latency}) \quad (2)$$

As we can see with the formula, assuming our lower-level caches (L1, L2) don't have abysmal hit rates, higher-level caches suffer from diminishing returns. This is why we don't really see L5, L6 caches.

Four considerations when designing memory systems

- **Placement** - where do we keep the data within the structure?
- **Indexing** - how do we read data out of the structure?
- **Replacement** - cache size $\ll$ main memory size, so how do we replace old data?
- **Writes** - writes actually change architectural state

3.1 Cache Organization

- consists of a tag array and a data array (implementation wise, data array is a big SRAM block, tag array can be SRAM or flip-flops)
- cache consists of sets, each set having N blocks where N is the degree of associativity
 - direct-mapped cache means $N = 1$
 - K least-significant bits ($K = \log_2(Q)$ where Q is the block size) are used to identify specified bytes within each block
 - next M bits ($M = \log_2(S)$ where S is the number of sets) is the index.
 - remaining upper bits are reserved for tag
- Different addresses can map to the same data array, so we need to maintain the tag array in order to map entries back to memory addresses.
- Cache access can often be the critical path in a processor design—there's value in pipelining it (separate into address calculation/disambiguation/access stages)
 - in this pipelined design, we can either access tag array/data array in parallel or sequentially (in different pipeline stages)
 - parallel implementation has higher power consumption, lower max clock freq., but is also one clock cycle faster
 - use parallel for in-order processors which can't hide latency well, and serial for out-of-order (OoO) processors which can.
- Direct-mapped caches are fast, but suffer more conflict misses than associative caches. Higher associativity means less conflict misses, but also larger muxes and more access latency.
- Modern processors have separate instruction and data caches.
 - Data size is often much larger than instructions, so a unified L1 data & instruction cache would be dominated by data.
 - Instructions often have more locality than data. e.g. consider large graph data structures.
- Data caches should be non-blocking, instruction caches should be blocking and single-ported.

Miss Classification

- **Compulsory Miss**: “cold start miss” - misses due to data not being cached yet.
- **Capacity Miss**: not enough space in cache to hold data. e.g. L1 cache is 32KB, you cannot physically fit any data structure larger than 32KB.

- **Conflict Miss:** data in cache misses because it was replaced. e.g. L1 cache is 32KB, you want two arrays of size 1KB to be both be cached, but they map to the same set, and so one replaces the other within the cache.
 - **Thrashing** - When frequently used data is consistently evicted → re-loaded into cache.

Direct Mapped Cache

- A memory value has a single corresponding location in the cache
- Rather simple to implement, but suffers from conflict misses

Fully-Associative Cache

- Memory value can be placed anywhere in the cache
- 0 indexing bits
- Power, Performance issues—we need comparators for each way in the fully-associative cache.

Set-Associative Cache

- A memory value can be placed in any location within its set.
- A good compromise between DMC and FAC, most cache systems nowadays are set-associative.

Writes

- Write-back: add dirty bit, if a line is evicted and marked as “dirty,” write to memory.
- Write-through: on every write operation, write to main memory.

Imagine a system with L1/L2 cache hierarchy. Write operation misses in L1, hits in L2:

- Write Allocate Cache: on writes, load data into L1 cache and then modify it.
 - pairs well with write-back
- No Write Allocate: on writes, load data directly into processor, don’t allocate anything in L1.

Replacement Policies

- For direct-mapped cache, its fairly trivial to determine what data to replace. Becomes less trivial with set-associative/fully-associative caches.
- LRU (least-recently used) tends to be the preferred replacement policy because it assumes that the recent past can predict near-future outcomes, pairing well with the idea of temporal locality.
 - not **always** optimal—e.g. streaming data with less re-use.

LRU Implementation:

- for each cache line, maintain a queue containing N entries, where N is the number of ways/ associativity. Each entry has size $\log N$
- on a memory access that hits in the set, push the selected way to the front of the queue. The back of the queue will be the LRU.
- this is viable in a 2-way set-associative cache (especially because you can do it with a single LRU bit)
- scales poorly for anything that isn’t 2-way.

pLRU

- used in ≥ 2 -way set-associative caches
- Use a binary tree where each node is a single bit determining which subtree is LRU.
- called “pLRU” because its not strictly LRU, but approximates it well.
- guarantees we won’t evict MRU (most-recently used)
- For a N -way set associative cache, pLRU requires $N - 1$ bits. This scales a lot better than $N \log N$.

Lockup-Free/Non-Blocking Caches

- Blocking caches will stall the processor until a request is serviced.
- Executing instructions while cache misses are serviced requires dependency-tracking mechanisms, such as those in OoO processors.
- **Lockup-Free/Non-Blocking** caches allow load/store operations to be issued, despite pending cache misses; one method of implementing this is through **Miss Status Holding Registers (MSHRs)**
 - MSHRs hold information about pending misses

3.1.0.1 Implicitly vs. Explicitly Addressed MSHRs

- Implicitly-Addressed MSHRs
 - simplest design (low area/power consumption)
 - each MSHR holds data array block address and a valid bit (set on a primary/compulsory miss)
 - comparator to match future (secondary) misses of a block to the same MSHR
 - each cache block consists of N 32-bit or 64-bit words, so the MSHR contains N entries to record miss information (e.g. destination register of the instruction, size of load instruction)
 - can only support **one** outstanding miss per word because the block offset is implied by the position within the MSHR
- Explicitly-Addressed MSHRs
 - add block offset to each MSHR field–position within MSHR no longer implies address
 - we can have an arbitrarily high number of outstanding misses for a single block, without causing structural stalls

3.1.0.2 In-Cache MSHRs

- instead of allocating separate SRAM/register arrays for MSHRs, we make use of pre-existing cache resources
- requires one more bit to be added to the tag array–the transient bit–to indicate that a block is currently being fetched
- when in transient mode, the tag holds the block address, and the data array holds MSHR information
- can either be implicitly or explicitly addressed

3.1.0.3 Multiported Caches

- Can we do >1 load/store instruction per cycle?

- The naive implementation: doubling all control/data paths (double way muxes, double tag comparators, double aligners, etc.)
 - significantly increases cache cycle time
 - doubling tag/data arrays means changes/accesses to one cache needs to be broadcast to the other
- Virtual Multiporting involves issuing a memory operation on both the rising and falling edge of a clock cycle
 - does not scale well to higher frequencies
- **Multibanking**
 - divide cache into smaller banks, each of which are single ported.
 - multiple instructions can be issued per cycle assuming they access separate banks (otherwise, you end up with bank conflicts)
 - does not require tag/data arrays to have multiple ports
 - probably the best method of emulating multiported caches today

3.2 Cache - Virtual Memory Interactions

Quick Review of Virtual Memory

- **Virtual Memory** - “virtualize” memory, gives the illusion of a larger continuous logical address space.
 - only a small part of the program needs to be in (physical) memory at a given time in a program.
 - We can have synonyms, virtual addresses of different addresses mapping to the same physical address, allowing for better utilization of memory for shared resources like libraries.
 - Allows for efficient process creation.
 - Paging vs. Segmentation: paging involves dividing the logical address space into fixed-size pages, whereas segmentation uses variable-sized segments which represents a part of the program (e.g. call stack)
 - Advantages of Virtual Memory
 - **Translation:** programmer gets a consistent, continuous view of memory.
 - Only the most important part of the program is loaded into physical memory.
- Protection:** different processes have different address spaces, and are thus protected from each other.
- Different pages can have different protections (some can only be accessed by supervisor status, etc.)

3.2.1 Address Translation

How do we map back and forth between physical and virtual addresses?

- The processor has a paging mechanism to virtualize the linear physical address space, which involves dividing the address space into pages (can lie in main memory or external storage), and it keeps a mapping of virtual to physical pages in the page table (main memory).
- Virtual Address consists of {VPN, offset}, Physical Address consists of {PPN, offset}.
 - ▶ First we need to find the physical address of the page table entry (PTE)
 - PhysAddr of PTE = $VPN \times \text{Size of PTE} + \text{PTBR}$ (Page Table Base Register)
 - ▶ Then we issue the memory request to get the PTE, which contains our PPN.
 - ▶ Concatenate PPN with offset (same offset in Vmem and Pmem) to determine physical address.
- A one-to-one mapping of PPN $\Leftrightarrow$ VPN is sometimes not possible (not enough space in RAM to store all mappings)
 - ▶ Solution: Multi-level page tables. Each virtual address now indexes {PT1, PT2, ..., offset}
 - ▶ Worst case scenario, a process uses up its entire virtual address space and you have to store extra info for the L1, L2, ..., mappings.
 - ▶ More realistically, a process uses its virtual address space rather sparingly, and the multi-level page table saves you from having to allocate large continuous memory chunks that end up not being used.
 - Allows for fragmentation of the page table in physical memory.
 - ▶ Major Downside: now you need extra memory “traversals” just to find a singular PPN.
- Instead of multiple memory accesses to decode addresses, we use a **translation lookaside buffer (TLB)**, which is effectively a page table cache.
 - ▶ ~ 1 cycle access time, indexed by page number of virtual address, returns physical page number
 - ▶ associativity and size varies (can have tens or hundreds of entries)
 - ▶ TLBs have amazing hit rates (99-99.9% hit rates, usually)
- Program data/instruction addresses are usually **virtual addresses**. You can have virtually-indexed, virtually-tagged (VIVT) caches, but they suffer from aliasing (multiple processes sharing virtual addresses).
 - ▶ **Homonym problem** (one virtual address maps to different physical addresses)
 - ▶ **Synonym problem** (multiple virtual addresses map to the same physical address)
 - ▶ VIVT suffers from **both** problems.
- To avoid aliasing/homonym problem, we physically tag addresses (VIPT/PIPT)
 - ▶ while PIPT avoids homonyms, its also suffers from slow memory lookups as it needs to go through address translation each time.
- Nuance 1: **Synonym Problem isn't inevitable for VIPT caches**
 - ▶ If $\# \text{ sets} \leq \frac{\text{Page Size}}{\text{TLB Cacheline Size}}$, then we're guaranteed to not have the synonym problem.
 - Virtual Address can be broken up into {VPN, offset}

- When we're addressing into the TLB, our address is broken up into {TLB tag, TLB index, block offset}
- Two different virtual addresses which alias to the same physical address **will** share page offsets.
- If TLB index and block offset can be contained s.t. its less than or equal to the size of the page offset, our indexing scheme never "touches" VPN
- Thus when we have two virtual addresses which alias and are inputs to the TLB, they will index into the same entry, and the tag check will be done with respect to physical addresses.
- **Case Study: MIPS R10000**
 - Contains L1 (VIPT) and a direct-mapped L2 TLB (PIPT) which is inclusive of the L1
 - Indexing "bleeds" into VPN, thus causing aliasing in L1 TLB.
 - The segment which "bleeds" into VPN is appended to tag in the L2 TLB. Note: tag entry of L2 contains physical address tag + virtual address bits from VPN
 - Assume two addresses A (which already has an entry in L1/L2) and B which is a synonym of A.
 - B indexes into L1 TLB, goes to a different index in L1, cache miss.
 - B is translated into a physical address, goes to L2 cache, goes to same block as A, but fails because the lower bits of the VPNs don't match.
 - This triggers a conflict miss, A's entry in L1 is ejected and replaced with L2.
 - We eliminate the synonym problem as only one of these synonyms have a mapping in L1 at any given point in time.
- Nuance 2: **What do we do on a TLB miss?**
 - We have the option to handle TLB misses in hardware (x86 does this), or software by raising a TRAP and having the OS take care of it.

4 Out-Of-Order Execution

4.1 Core Components and Stages

4.1.0.1 Allocation

- This phase handles register renaming and instruction dispatch.
- Instruction Dispatch reserves resources that will be later used by the instruction include:
 - Entries in the Issue Queue
 - Entries in the Re-order Buffer
 - Entries in the Load/Store Queue
- In the case that there are multiple issue queues associated with multiple functional units (FUs), allocation should determine which FU to map an instruction to.

4.1.0.2 Register Renaming

- Register Renaming is specific to OoO architectures, used as a method to take advantage of Instruction-Level Parallelism (ILP).
 - There are two types of dependencies between instructions: data dependencies (RAW) and name dependencies (WAR, WAW)
 - Data dependencies must be in-order (nothing we can do about that)
 - Name dependencies, on the other hand, are *false* dependencies—only an issue because of constrained storage.
 - Thus, a potential solution is to have (issued) instructions write in a different storage location
 - There are huge performance benefits associated with Register Renaming, due to the limited register file size of modern processors (e.g. 32 integer registers for RV32I) leading to a high prevalence of dependencies.
- Tomasulo implemented register renaming using the identifier of the reservation station (RS) of the destination of the instruction. This isn't really done today.

Register Renaming via the Re-order Buffer

- ROB is basically a FIFO structure, with tail allocated for each new instruction, and head released on instruction commit.
- You have a Register Rename Table, **Architectural Register File (ARF, specified by ISA)**, and **ROB** (in a later backend pipeline stage)
- ROB contains results of uncommitted instructions, ARF contains latest committed results
- For each entry in the rename table, whose size is fixed with respect to ARF, we have information on whether latest definition is in ARF or ROB, as well as the position within ROB (latter case).
- During the commit stage, the value is copied ROB → ARF

Register Renaming via the Rename Buffer

- Downside of Register Renaming in ROB is that instructions that don't produce a register result take up unnecessary space in the ROB.
- Separate structure (Rename Buffer) for results of in-flight (uncommitted) instructions. This way, only instructions which write back to registers take up storage space.
- Otherwise, implementation is similar to Register Renaming via ROB.

4.1.0.3 Merged Register File

- Single register file holding both committed and speculated values. Thus, size is larger than ARF.
- Each register is either free or allocated.
 - Free registers are stored in a free list (circular buffer storing all identifiers)
 - Allocated registers can either store committed or uncommitted/speculative data
- **Register Alias Table (RAT)/ Register Map Table** - maps between physical register file (PRF) and ARF.

- When an instruction is renamed, we look at the RAT to find the source operands; if the instruction writes to a register, we allocate a register on the free list—if this isn't possible, we need to stall.
 - The destination operand is renamed to this new free register, and the RAT is updated accordingly.
 - Physical registers are freed when the following instruction which shares the same **architectural** destination register commits.

Register File Reads

- We can either *read before issue* or *read after issue*.
- If we read before issue, we selectively read available values, and unavailable operands are marked as such in the issue queue, and later obtained via data bypassing.
 - Allows for a fewer number of ports in the register file, comes at the cost of larger issue queues (area/power)
- If we read after queue, issue queue stores identifiers of register source operands, and the register file is actually read once the instruction is issued to be executed. Once again, we use data bypassing for uncommitted results.
 - Requires more ports in the register file, but requires operands to be read only once.
- ROB/Rename Table-based renaming works well with read before issue. Merged Register File-based renaming works well with read after issue.

4.1.0.4 Issue Stage

- In the case of an in-order processor, a typical implementation of the issue stage would just use scoreboarding:
 - Data dependence table which keeps track if operands are unavailable, or available (can either be sent from some bypass level, or available in register file)
 - Resource availability table which keeps track if functional units are available.
 - Tables indexed using source register identifiers of the instruction to be issued.
- In an out-of-order processor, instructions sit in Issue Queues/Reservation Stations, waiting for CDB (common data bus) to broadcast speculative register values.
 - Once all operands of an instruction are ready, it can be steered for execution.
 - Multiple instructions in a reservation station can be available at once, and so its advantageous to have a good scheduling policy to select which instruction to select.
 - We can prioritize “older instructions” (age-ordered queue), or “problematic” instructions (instructions with a high degree of dependency)
- Aside: “waking up” and selecting instructions for execution in the reservation stations/issue queues involves a lot of CAM matching, which often lies on the critical path. Thus, its often advantageous to separate them out into separate pipeline stages.

4.2 Tomasulo's Algorithm (with ROB)

Core Idea:

- Issue an instruction in the instruction queue to its RS if there's an available slot. Destination is renamed to enqueued ROB ID.
 - Operands read from either register file or ROB
- When operands are ready, steer the instruction for execution. The results of the instruction are written on the CDB;
 - If not ready, snoop CDB. If CDB broadcasts the renamed id (ROB ID) that happens to the operand, write that value into the RS entry, mark that operand as ready.
- After being executed, as part of the CDB broadcast, broadcast all results to all RS and write the speculative value into ROB.
 - Mark the old RS entry as available.
- When instruction reaches the top of the ROB head, dequeue it, write the new register value into register file.
- **Pros:** It does what it needs to do—eliminates WAW, WAR dependencies through register renaming
- **Cons:** Result of functional units stored in RS, ROB, and Register File simultaneously (inefficient)
- These problems associated with Tomasulo's Algorithm are often solved through **ERR (Explicit Register Rename)** and register file read-after-issue.
 - Would go in-depth about ERR, but this is just the basis of mp ooo

4.3 Precise Exceptions

- **Exceptions** - internal problems in the execution of a program.
 - Ex: divide by zero, access protections, page fault (sometimes), etc.
- **Interrupts** - external events in the execution of a program.
 - I/O devices, power failure, etc.

Both require:

- the processor to halt (sometimes temporarily)
- saving processor architectural state
- handling the exception/interrupt (you need some handler)

When do we handle them?:

- We only handle exceptions when it's known (and known to be not speculative)
- Interrupts: it depends, sometimes we don't need to handle it right away if it's low-priority

- **Precise Exceptions:** The architectural state should be consistent when the exception is ready to be handled.
 - All previous instructions should be committed, all following instructions shouldn't

Why do we want precise exceptions?

- Semantics
- Allows for a better debugging experience as a programmer (its easier to determine what caused the exception)
- Allows easy recovery from the exception
- Enables traps into software
 - Ex: Software-defined opcodes: undefined opcode exception goes into a custom handler/subroutine.

5 Memory Systems

5.1 DRAM

- **Dynamic Random Access Memory** - uses capacitors to store data.
 - “Dynamic” means you need you refresh the DRAM cell every couple ms or so.
 - Destructive reads, as the capacitor discharges.
 - Capacitors are small, and thus $Q = CV$, the charges are small as well.
 - You need sense amplification to read well.
- Organized in a hierarchy: Channel → DIMM → Rank → Chip → Bank → Row → Column/Cells
 - DRAM reads read an entire row into a row buffer, which then goes through a column decoder to return the specific sub-portion of the row.

5.1.0.1 DRAM Policies

Three DRAM operations

- ACT: open a row
- CAS: perform R/W operation on the opened row (column access strobe)
- PRE: close the row and precharge

Row Policies

Open: Assumes next DRAM request will be in the same row, keeps data in row buffer.

- Best Case: next DRAM request reads from same row, don't need to interact with DRAM bank
- Worst Case: next DRAM request is to another row, need to do PRE, ACT, and CAS

Closed: Assumes next DRAM request will be in another row—writes data from row buffer back to the bank.

- Alleviates worst case: regardless of the next request, you just need to do ACT and CAS

Scheduling Policies

- FCFS: naive, retrieve whatever address you get first.
- FRFCFS: look at FIFO, if you have a row hit, retrieve that data out-of-order
 - good for MLP, temporal locality

5.2 Load-Store Queue (LSQ)

- The most basic implementation of a LSQ is a in-order FIFO
 - Memory operations are issued only when it hits the top of ROB
 - Simple and requires no “recovery/rollback” logic, but doesn’t exploit ILP, MLP, and kinda bad for performance.
- Store-to-load forwarding: Check LSQ for older stores to the same address, forward store data and execute loads out-of-order.

Split-LSQ and Post-Commit Store Buffers

- Separate load and store queues allow better forwarding within a core, drains to cache on retirement.
- On commit, loads can be moved to a post-commit store buffer, where they are globally visible, and eventually write to L1D cache.
 - Decouples commit from write-port availability
 - Enables memory coalescing into a single cache line (imagine memcpy and memset)
 - Allows cache to busy with loads
 - Allows store queue to drain faster (less backpressure)
- Aside: **FENCE** instructions drain the LSQ, preventing newer memory instructions from entering the LSQ until its empty.
 - **FENCE.I** is a thing in RISC-V which also flushes L1I, allowing for things like self-modifying code.
- **Aggressive memory re-ordering and forwarding has an associated power/area cost.**

6 Multicore Systems and Multithreading

Why can't we just increase issue width, pipeline depth, and clock frequency?



Figure 1: Them hardware engineers in Austin—iykyk

We can derive a rough function for the optimal number of pipeline stages in a processor (P_{opt})

$$P_{\text{opt}} = \sqrt{\frac{N_I t_p}{\alpha \gamma N_H t_D}} \quad (3)$$

where: N_I is number of instructions, t_D is the latch delay, N_H is the number of hazards, α is average degree of superscalar processing, γ is a factor which represents the average “impact” of a hazard.

This formula helps identify tradeoffs:

- e.g. Better branch prediction -> less hazards -> we can increase pipeline depth
- Larger superscalar fetch -> α increases -> we need to decrease pipeline depth
- **This formula does not take into account power**

$$P_{\text{dyn}} = n C V^2 f \alpha \quad (4)$$

In this case, α is the activity factor. We're also assuming $P_{\text{static}} \ll P_{\text{dyn}}$

- Factoring this into the equation, we get an ugly **polynomial**
- In other words, power will blow up the moment you do deep pipelines and push your processor to higher and higher clock frequencies.

Imagine a system with N transistors

- If it were all dedicated towards a single core, you have $O(N)$ power and area, and $O(\sqrt{N})$ performance
- If it were dedicated towards k multi-cores, then you have $k * \times O(\frac{N}{k}) = O(N)$ total power and area, and $O(k \times \sqrt{\frac{N}{k}}) = O(\sqrt{k}\sqrt{N})$ performance.

6.1 Multi-Core Systems

Mandatory Amdahl's Law Mention - It always comes up

$$\text{speedup} = \frac{1}{(1 - f) + \frac{f}{p}} \quad (5)$$

where f is the % of the program which is parallelizable, and p is the number of parallel processors.

Gustafson's Law - You can scale up the size of a parallelizable program with respect to the number of parallel processors.

Shared vs. Distributed Memory

- Shared memory interfaces are great from a programmer perspective, but introduces race conditions between threads, and less "safety" memory-wise.
- Distributed Memory is less convenient to work with, but requires explicit messaging/communication between threads.

Interconnect Topologies

- Shared bus and arbitration (low bandwidth, low area/power)
- Crossbar connection (high bandwidth, high area/power)
- Mesh networks, NoC-based communication, etc.

Cache Coherence

- How do you ensure each core gets the most recent data?
- How do memory load/stores propagate to other cores?
- Very important!!!!

6.2 Multithreading

- Instead of multiple cores, you can support a single core with multiple threads of execution
 - A thread is just a PC, associated regfile/architectural state

- duplicating register file, multiplexing memory, often times data is transferred from a secondary regfile into a primary regfile for execution.
- **Coarse-Grained Multithreading (CGMT)**: switches threads on major stalls, fast single-thread speed.
- **Fine-Grained Multithreading (FGMT)**: switches threads every clock cycle, fast switching between threads.

7 Memory Consistency and Cache Coherence

7.0.1 Program Order & Memory Consistency

Weak Ordering

- Divide operations into data operations and synchronization primitives
- e.g. fence operations
 - all memory ops before the fence must complete before fence is executed
 - all memory ops after the fence must wait for the fence to complete (drains LSQ)
 - fences are performed in program order

Release Consistency

- synchronization divided into acquire (lock) and release (unlock)
- acquire must complete before all following memory ops

Sequential Consistency

- imagine a mux/switch which only grants one core to memory at a given time
- very good for defining “correctness,” but bad from a performance perspective because we cant use non-blocking caches, write-back caches, and aggressive memory re-ordering.

Weak(er) Consistency

- ordering of operations only important when order effects operations on shared data
- fence instructions
- allows for higher performance (programmer-specified regions of atomic operations), but added burden on programmer

Shared Memory Synchronization

- mutexes
- point-to-point synchronization (producer-consumer)
- rendezvous (barrier synchronization), allows for **bulk synchronous programming**

Locks

- reliable locking requires atomic read-modify-write instructions
 - e.g. test & set
- sub-atomic Locks
 - use two instructions (load linked + store conditional (LL + SC))

- ▶ Load Linked:
 - reads mem. value
 - sets flag
 - writes address to special global address register
- ▶ Store Conditional:
 - writes value if flag is set
 - no-op if flag is cleared
 - sets CC indicating success/failure

7.1 Cache Coherence

- coherence is the ordering of operations from different cores to the same memory location whereas consistency is about the ordering of all memory operations

7.1.1 Snoopy Bus versus Directory-based

Snoopy Bus

- single point of serialization for all memory requests
- processors observe other processors' R/W
- straight-forward to implement, doesn't scale very well though

Directory-Based

- Single point of serialization per block
- processors make explicit requests for a block, directory tracks which caches have each block
- locally central directory keeps track of where the copies of each cache block reside. P cores, $P + 1$ bits for directory
 - ▶ on a read, set cache bit and arrange the supply of data
 - ▶ on a write, invalidate all caches that have the block and reset their bits
- exclusive bit = "single source of truth" (producer, effectively)

7.1.2 MSI

- **(M)odified** cacheline is the only cached copy AND dirty
- **(S)hared** cacheline is one of potentially several cached copies and is clean
- **(I)nvvalid** cacheline is not present in this cache

Fairly simplified cache coherency scheme, but has a few issues:

- a newly fetched block immediately goes to Shared state, even if it's the only shared copy
- if a processor writes into this sole copy, it broadcasts an invalidate, even though it's unnecessary
- not great for cache-to-cache transfers

7.1.3 MESI

- **(E)xclusive** cacheline is clean, but only I have it.
- transition from Exclusive or Modified state to shared is called a "downgrade" because it takes away the owner's right to modify data without having to invalidate

Better than MSI, but has a few issues:

- downgrades are bad for performance (has to write back to main memory), especially bad if cacheline is getting ping-ponged between multiple processors, each of which are repeatedly downgrading/upgrading
- bad for cache-to-cache transfers

7.1.4 MOESI

- **(O)wned** cacheline is dirty and shared, this processor is the sole source of truth for this cacheline.
 - only a BusWr on Owned will invalidate this cacheline

there are other states you can add to your CC protocol, but it has its tradeoffs with respect to hardware complexity, bandwidth, etc.

Multi-level Cache Hierarchies

- inclusion property, L2 includes L1. If marked modified/shared in L1, it should also be marked in L2
- only need to snoop for L2
- L1 should access L2 for misses/block state changes
- L2 forwards L1 blocks invalidated/updated by the bus.

8 Miscellaneous

8.1 Interconnects

NoC (Network-on-Chip)

- ad-hoc wiring does not scale
- NoCs offer efficient multiplexing of data packets

Metrics for Evaluating NoCs

- Switch Degree - number of links at a node (good proxy for cost)
- Hop Count - average number of “hops” data needs to travel from one node to another
- Maximum Channel Load - bandwidth, effectively
- Bisection Bandwidth - split graph in half, in a uniform NoC, half of traffic should pass Bisection
 - proxy for cost, global wiring
- Path Diversity - multiple shortest paths between source and destination
 - fault tolerance

Types of NoC Topologies

Crossbar - directly connects n inputs to m outputs (no intermediate stages)

- fully connected, hop count = 1
- routers often have internal crossbars

Bus

- all producers/consumers share a common shared Channel

k-ary n-cube

- k^N network nodes
- N-dimensional grid with k nodes in each N-dimension
- **Torus** is a common interconnect topology
 - edge symmetry is good for load balancing, we lose symmetry if we remove the wrap-around edge links

There are also butterfly, clos networks, etc.

- add an intermediate stage of switches to avoid the non-scalability of crossbar
- Folded clos/fat tree features a constant bandwidth of each level of the tree

There are also various routing algorithms through NoCs

- oblivious routing algorithms do not consider current network state
- adaptive routing algorithms exploits path diversity, and considers current network state

8.2 VLIW

- no register scoreboard, renaming, etc. Processors read whatever value is in register/bypass logic.
- compiler knows delays of instructions, handles all ILP (instead of the hardware)
- nops are inserted if the processor cannot find enough parallel instructions (can potentially take up a large amount of space in instruction memory)
- very simple pipelined logic, but places the burden on the compiler
 - common in AI hardware, often tightly coupled with systolic arrays
- hard to maintain compiler compatibility between different versions of the processor

8.3 Power

- $P_{\text{dyn}} = \alpha n C V_{\text{DD}}^2 f$
 - we can decrease α through sleep mode, clock gating
 - we can decrease C through smaller transistors and shorter wires
 - we can decrease V_{DD} to the lowest suitable voltage
 - we can decrease f to the lowest suitable frequency
- Reducing static power can be done by:
 - selecting lower V_t devices
 - keeping in mind thermal effects, body bias, etc. when doing physical design.
- Power is incredibly important (just as perf.), we need to optimize for power at each design stage